Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

IC-8063 Comercio Electrónico

Documentación externa - B2C

Estudiante(s):

Yosimar Antonio Montenegro Montenegro	2023147365
Bryan Londoño Marchena	2022233102
Dylan Rodríguez Chavarría	2023161347

Profesor a cargo:

Joss Pecou Johnson

II Semestre 2026

Flujo de Compra - General

Catálogo → Agregar producto → Carrito → Realizar compra (checkout)
→ paso Información → paso Entrega → paso Resumen
→ Confirmar pedido (crea la orden PENDING)
→ Página de pago (datos de tarjeta → Pagar)
→ Página de confirmación (orden CONFIRMED)

Cada orden tiene los siguientes estados:

- PENDING → (Pago aprobado) → CONFIRMED
    - → (Pago rechazado) → PENDING (Payment Status = DECLINED)
    - → (Error de servicio) → PENDING (Payment Status = ERROR)
    - → (Cancelado por user) → FAILED

Cálculo de montos:

- **Subtotal** = suma de los subtotales de cada línea (cantidad × precio unitario).
- **IVA (13 %)**: round(subtotal × 0.13)
- **Envío** | ₡80.833 × totalUnidades × 2 (flete promedio sede San José × factor de traslado).
- **Total** | subtotal + IVA + envío.

Servicio de Pago

El servicio busca simular la pasarela de pago sandboxm cada vez que el usuario toca el botón de Pagar la pagina del pago envia al servicio lo siguiente:

Campo	Tipo	Detalle
orderId	number	Número de orden
amount	number	Monto a cobrar
currency	“CRC”	Moneda
card	CardData	Datos de la tarjeta, dato compuesto

La solicitud se ve así:

```
{
  "orderId": 3,
  "amount": 1450833,
  "currency": "CRC",
  "card": {
    "holder": "COMPRADOR PRUEBA",
    "number": "4000 1111 1111 0001",
    "expiry": "12/30",
    "cvv": "123"
  }
}
```

La respuesta del servicio sigue lo siguiente en Promise PaymentResponse:

Campo	Tipo
transactionId	string
orderId	number
amount	number
currency	"CRC"
status	"PAYMENT_APPROVED" | "PAYMENT_DECLINED"
message	string
processedAt	string

En caso de que el servicio no esté disponible responde con code SERVICE_UNAVAILABLE con status PAYMENT_DECLINED con rechazos por negocio.

Los escenarios se generan de la siguiente manera:

Condición	Resultado	Efecto
Solicitud inválida	PaymentServiceError (INVALID_REQUEST)	Orden → paymentStatus: ERROR
Tarjeta no encontrada / vencimiento o CVV incorrectos	PAYMENT_DECLINED	Orden → paymentStatus: DECLINED
r < errorProbability	PaymentServiceError (SERVICE_UNAVAILABLE)	Orden → paymentStatus: ERROR
Saldo insuficiente	PAYMENT_DECLINED ("Fondos insuficientes")	Orden → paymentStatus: DECLINED
Saldo suficiente y sin fallo	PAYMENT_APPROVED	Orden → orderStatus: CONFIRMED, paymentStatus: APPROVED

Tarjetas Existentes (cards.json):


```
[
  {
    "label": "Tarjeta con fondos amplios",
    "holder": "COMPRADOR PRUEBA",
    "number": "4000111111110001",
    "expiry": "12/30",
    "cvv": "123",
    "balance": 1000000000,
    "errorProbability": 0.15
  },
  {
    "label": "Tarjeta con fondos medios",
    "holder": "COMPRADOR PRUEBA",
    "number": "4000111111110002",
    "expiry": "12/30",
    "cvv": "123",
    "balance": 25000000,
    "errorProbability": 0.15
  },
  {
    "label": "Tarjeta sin fondos suficientes",
    "holder": "COMPRADOR PRUEBA",
    "number": "4000111111110003",
    "expiry": "12/30",
    "cvv": "123",
    "balance": 500000,
    "errorProbability": 0.15
  },
  {
    "label": "Tarjeta con servicio caído",
    "holder": "COMPRADOR PRUEBA",
    "number": "4000111111110004",
    "expiry": "12/30",
    "cvv": "123",
    "balance": 1000000000,
    "errorProbability": 1
  }
]
```


Actualización de órdenes:

Resultado del servicio	Función que se aplica	Cambios en orden
PAYMENT_APPROVED	markPaymentApproved(orderNumber, transactionId)	orderStatus: CONFIRMED, paymentStatus: APPROVED, guarda transactionId, paymentAttempts + 1
PAYMENT_DECLINED	markPaymentDeclined(	paymentStatus:

	orderNumber)	DECLINED, paymentAttempts + 1 (la orden sigue PENDING)
PaymentServiceError	markPaymentError(orderNumber)	paymentStatus: ERROR, paymentAttempts + 1 (la orden sigue PENDING)
Usuario cancela la compra	markOrderFailed(orderNumber)	orderStatus: FAILED

Manejo de errores en la plataforma

Hay errores de validación en formularios de campos obligatorios, longitudes, formato de correo y teléfono, campos obligatorios en direcciones.

La tarjeta exige que no haya campos vacíos, se rechazan tarjetas por servicio de pagos si no existe la tarjeta.

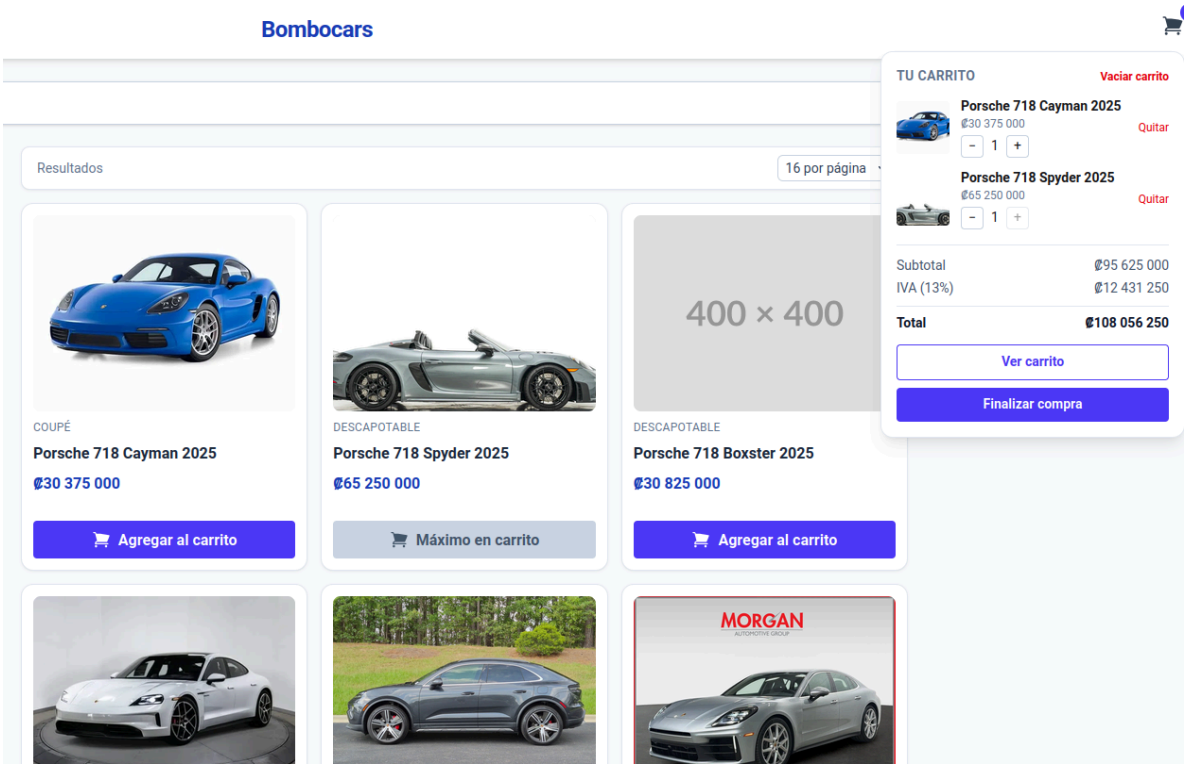
Hay errores de persistencia en caso de no poder guardar la orden, errores de render inesperados. Una página de 404 para rutas inexistentes.

Se previene doble envio de pago con estado isSubmitting mientras la orden se crea.

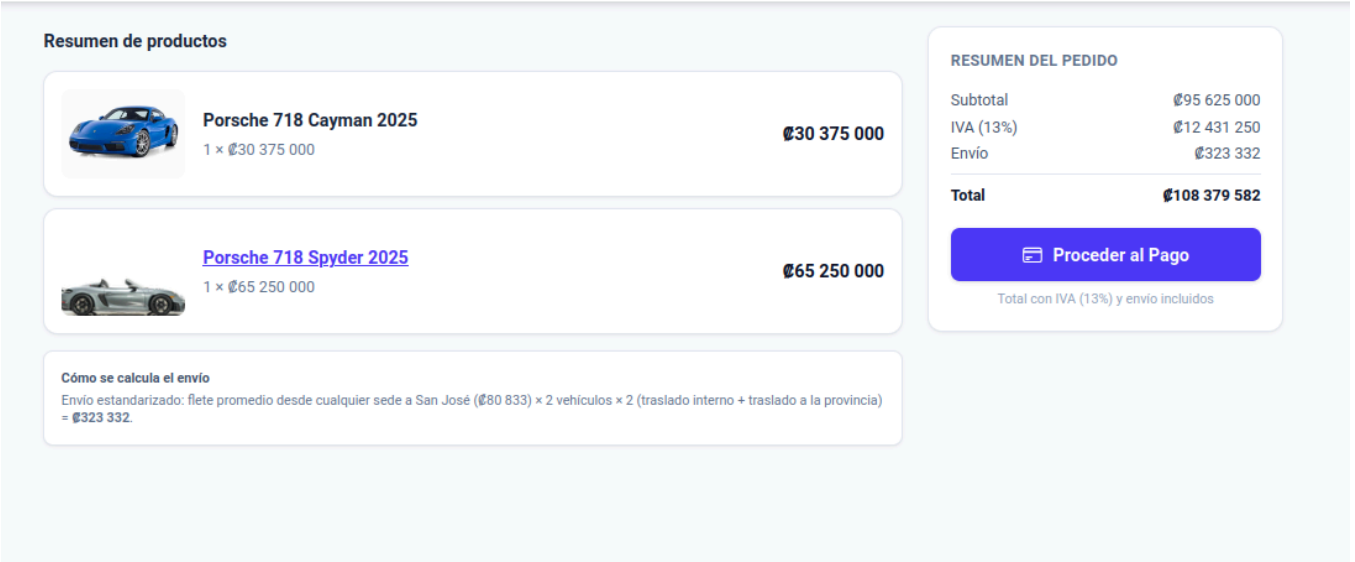
Pruebas de funcionamiento

Prueba de tarjeta con fondos suficientes:

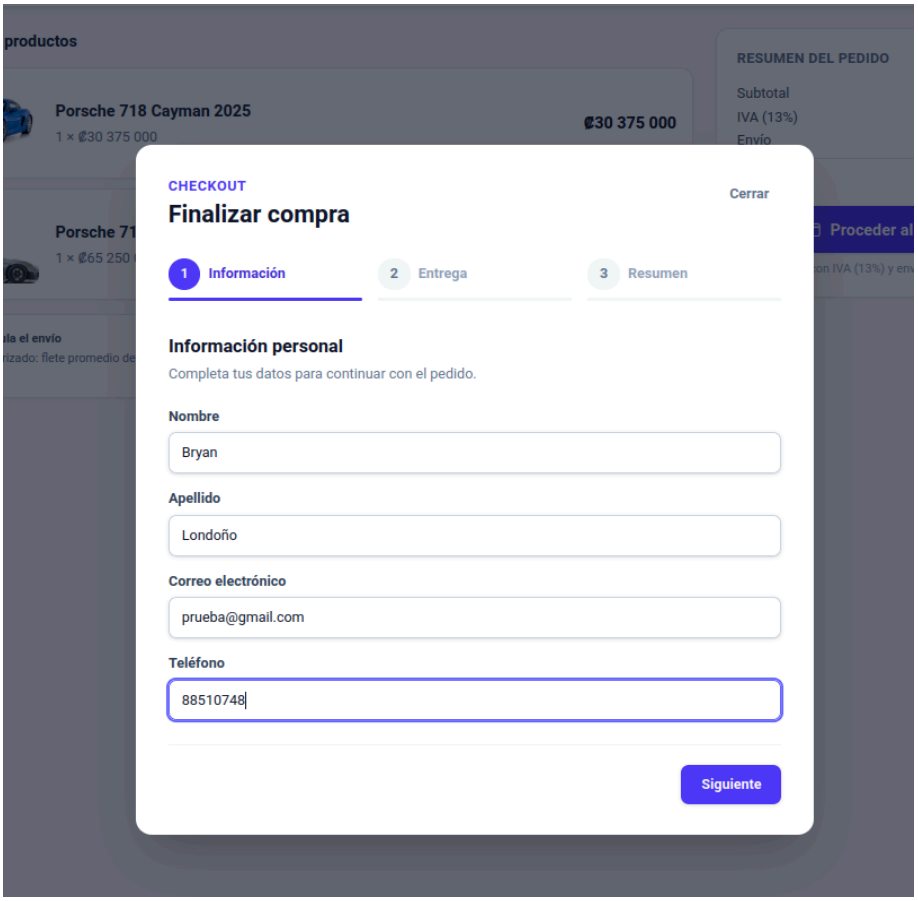
1.Seleccionamos los productos

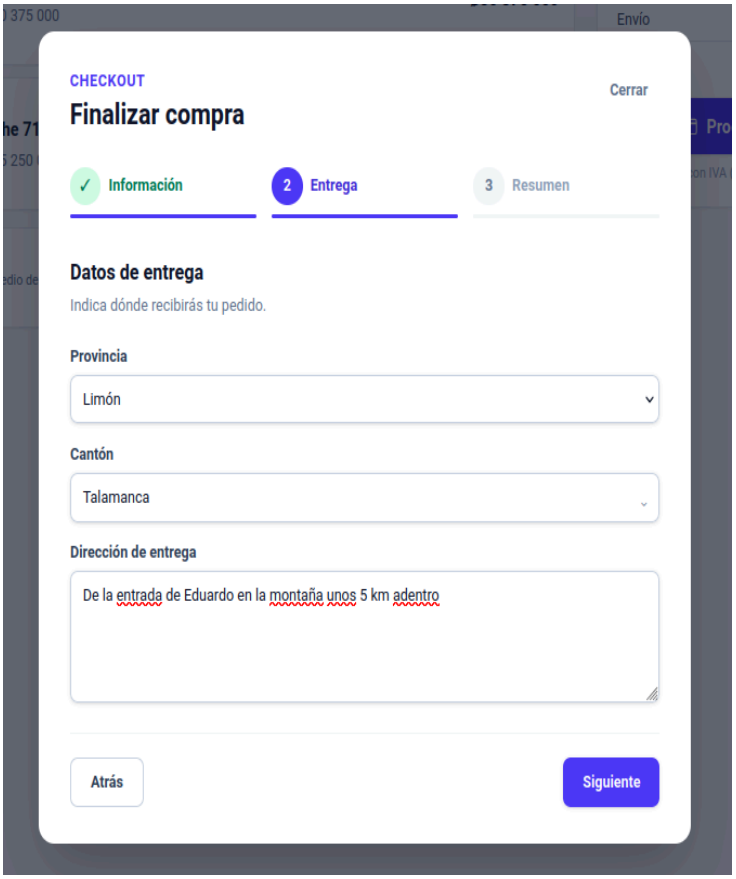


2. Procedemos a finalizar compra

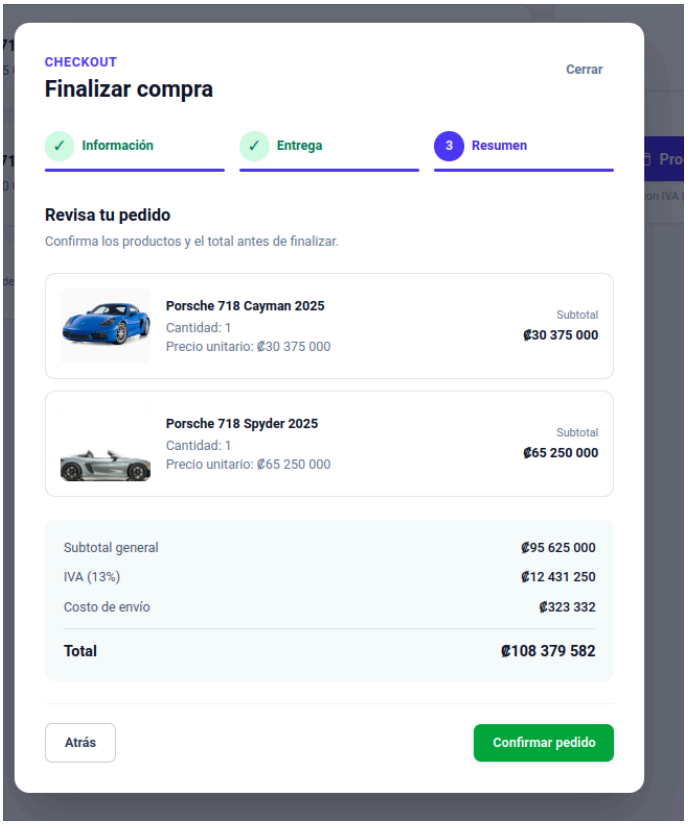


3.Rellenamos la información correspondiente

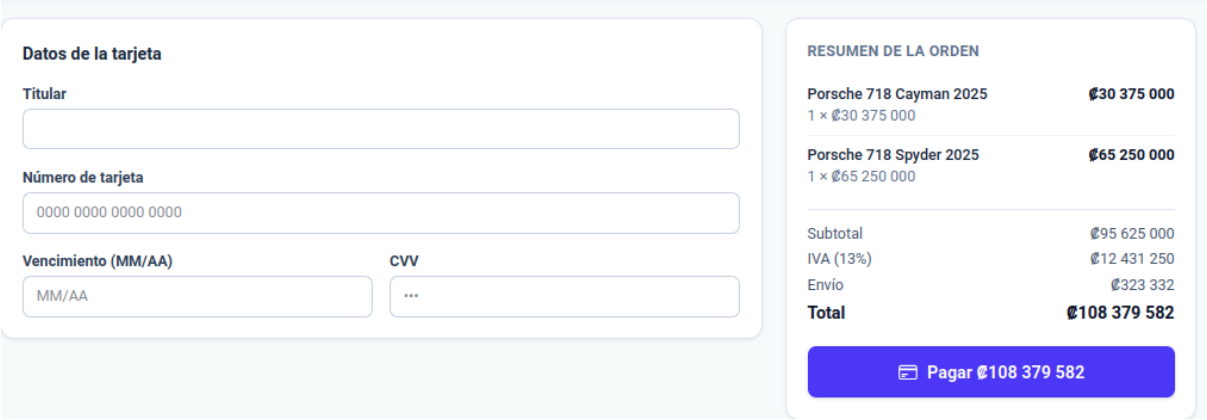




4. Verificamos los productos

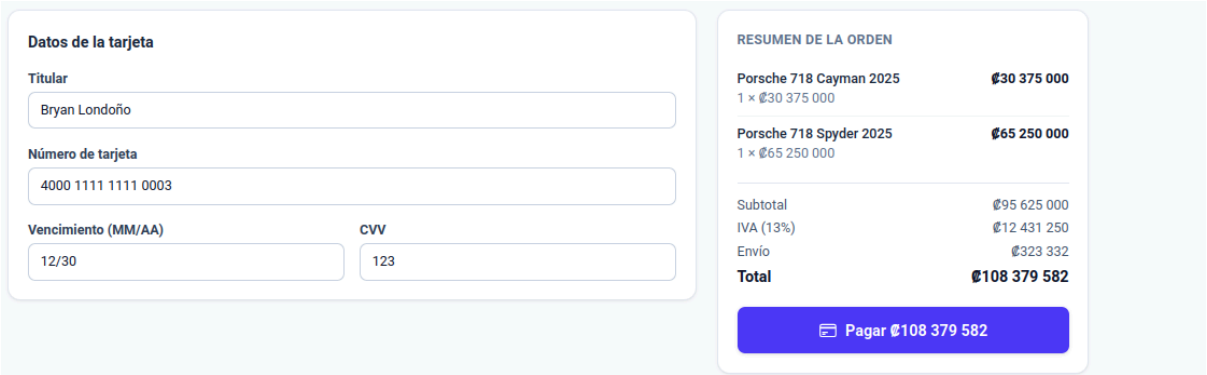


5. Presionamos confirmar pedido y rellenamos con los datos de la tarjeta a probar

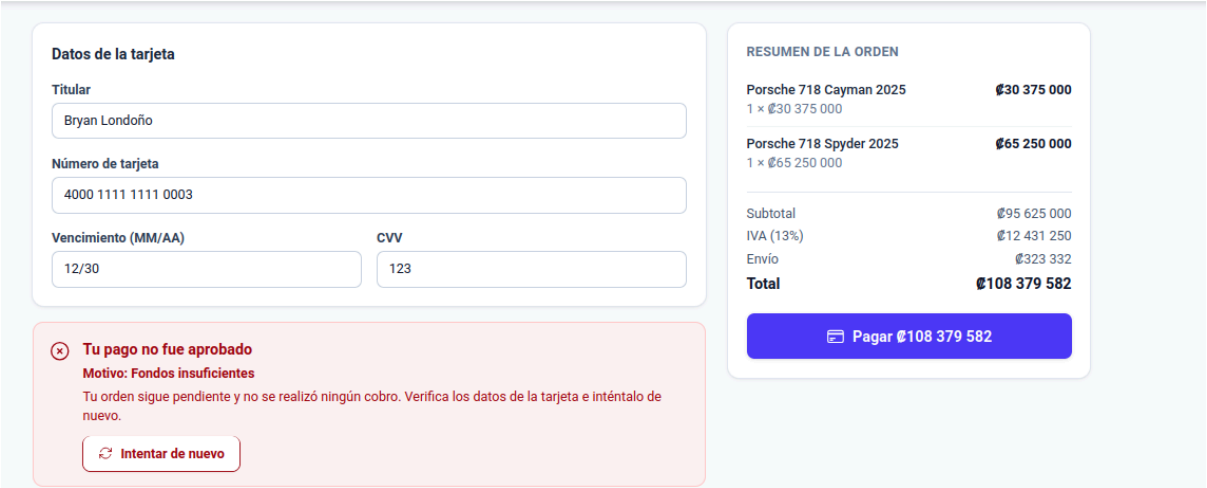


En este caso habrá 3 tipos de tarjetas para las pruebas

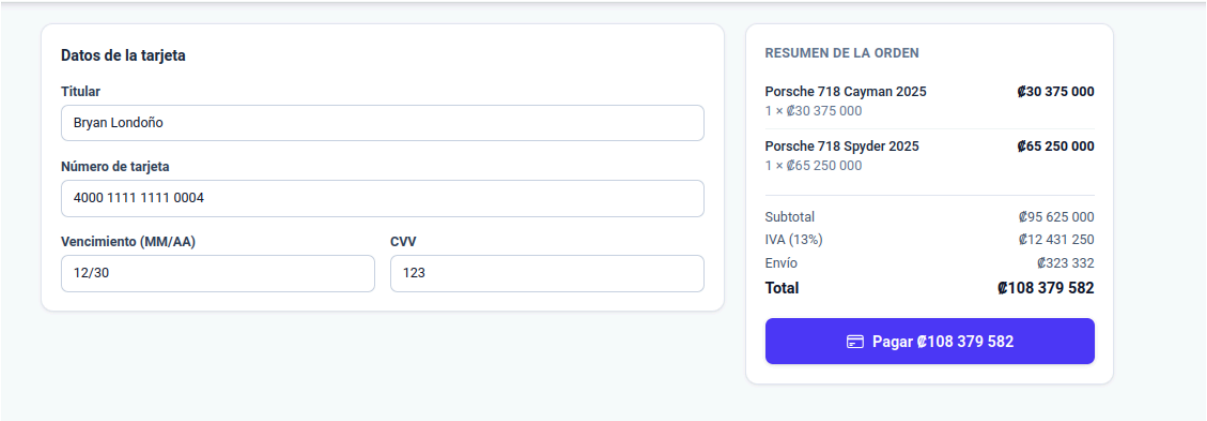
1. Prueba de tarjeta con fondos insuficientes



Presionamos el botón de pagar y nos debería mostrar lo siguiente



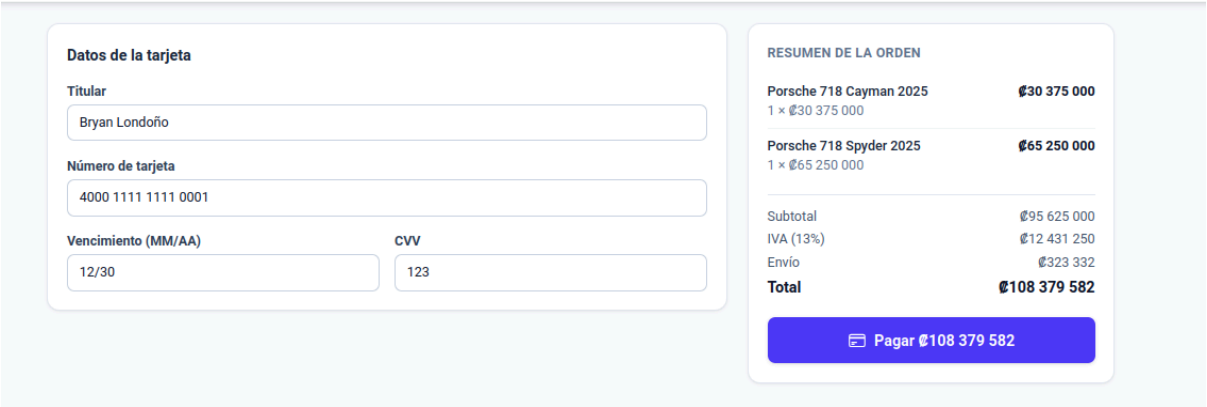
2. Prueba de tarjeta con servicios caídos



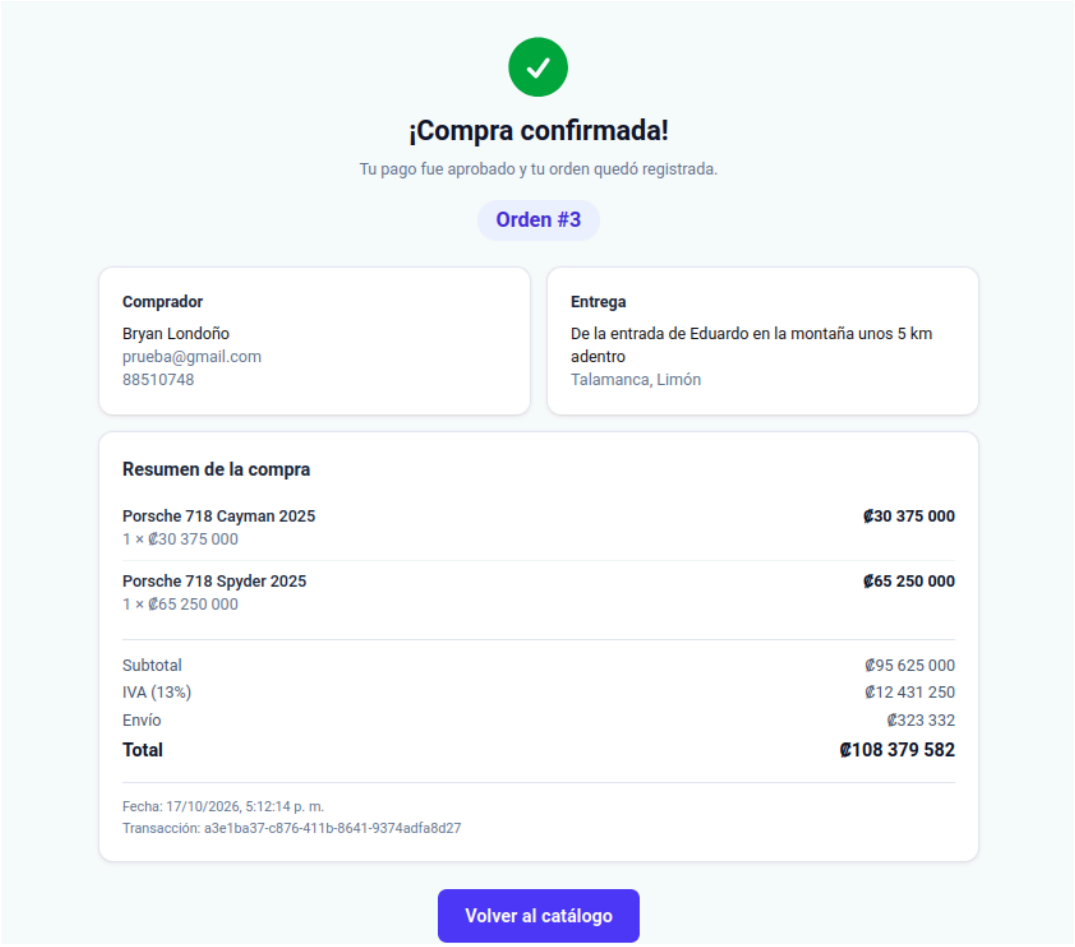
Presionamos el botón de pagar y nos debería mostrar el mensaje con el error del servicio de pagos



3. Prueba de tarjeta con fondos suficientes



Presionamos el botón de pagar y nos debería mostrar el mensaje indicándonos la información de la compra y que esta fue realizada con éxito.



Lecciones aprendidas

- DevOps: Configuración de entorno, compilación y deploy automatico en gh
- Creacion de servicios con promesas en request y response para temas de tarjetas y pagos.
- Validaciones de idempotencia y evitar transacciones dobles.

Enlaces

Repositorio de github:[Seraf1n0/PY1_Comercio-Electronico-Catalogo-Base-B2B]

URL de la aplicación: [b2b-front]